CS-104 Quiz 2 (ANSWER KEY)

Fall 2026

Name: _____

This is an in-class, closed-book quiz (30 points). If you have any questions, please feel free to ask them. When you are finished, you can turn it in and go.

1. (5 points)

Assume `x = 4`, `y = 9`, and `name = 'Ada'`. Give the value of each Python expression below (½ point each). If it raises an error instead, name the error.

```
>>> not True
```
False — example answer

```
>>> x > 3 and y < 9
```

```
>>> x > 3 or y < 9
```

```
>>> not x == 4
```

```
>>> 3 < x <= 4
```

```
>>> x % 2 == 0 and y % 2 == 0
```

```
>>> 'd' in name
```

```
>>> 'A' in name.lower()
```

```
>>> y == 4 or 5
```

```
>>> not (x > y)
```

```
>>> x + y > 12 and name == 'ada'
```

Answers: `False`, `True`, `False`, `True`, `False`, `True`, `False`, `5`, `True`, `False`.

2. (4 points)

What does this code print? Write every line of output.

```
total = 0
for i in range(1, 10, 3):
    if i % 2 == 0:
        total = total + i
    else:
        total = total - 1
    print(i, total)
print("done", total)
```

`1 -1`, `4 3`, `7 2`, `done 2` **(one line each; 1 point per line).**

3. (7 points)

Write a code segment that computes a shipping cost. It inputs a package weight in kg (as a float) and whether the shipping is express (the user types `yes` or `no`). Then:

- If the weight is 0 or less, print `invalid weight` and nothing else.
- Otherwise, the base cost is $5 for up to 1 kg, $9 for more than 1 kg and up to 5 kg, and for more than 5 kg it is $9 plus $2 for every kg above 5 (e.g. 7.5 kg costs 9 + 2 × 2.5 = $14).
- Express shipping adds $10 to the base cost.
- Print the final cost, e.g. `Cost: 24.0`.

Just write a code segment; you don't need to include documentation, an algorithm, or test cases.

```
weight = float(input("Weight (kg): "))
express = input("Express (yes/no): ")
if weight <= 0:
    print("invalid weight")
else:
    if weight <= 1:
        cost = 5
    elif weight <= 5:
        cost = 9
    else:
        cost = 9 + 2 * (weight - 5)
    if express == "yes":
        cost = cost + 10
    print("Cost:", cost)
```

Grading: inputs with conversion 1, invalid check 1, three weight ranges 3, express surcharge 1, output 1.

4. (2 points)

Fill in the blanks in each `range(...)` call so the loop prints exactly the numbers described (1 point each).

a) Every multiple of 7 from 7 to 70, inclusive:

```
for n in range(___, ___, ___):
    print(n)
```

b) The numbers 30, 27, 24, ..., 6, 3 (counting down by 3):

```
for n in range(___, ___, ___):
    print(n)
```

a) `range(7, 71, 7)` **(any stop from 71 to 77 works). b)** `range(30, 0, -3)` **(any stop from 0 to 2 works).**

5. (4 points)

Consider the following (syntactically correct) code segment:

```
count = 0
for i in range(10, 0, -2):
    count = count + i
print(count)
```

a) What does it print? b) Write a `while` loop that does the same thing.

a) `30` (1 point). b) 3 points:

```
count = 0
i = 10
while i > 0:
    count = count + i
    i = i - 2
print(count)
```

6. (8 points)

Write a code segment that repeatedly inputs whole numbers from the user until the user enters `0`. Then it prints how many numbers were entered (not counting the 0), their sum, and the largest one. Do **not** use the built-in `max` function or a list. If no numbers were entered before the 0, print `no numbers` instead. Example run:

```
Number: 4
Number: 11
Number: -2
Number: 0
Count: 3, sum: 13, largest: 11
```

Just write a code segment; you don't need to include documentation, an algorithm, or test cases.

```
count = 0
total = 0
largest = None
n = int(input("Number: "))
while n != 0:
    count = count + 1
    total = total + n
    if largest is None or n > largest:
        largest = n
    n = int(input("Number: "))
if count == 0:
    print("no numbers")
else:
    print("Count:", count, "sum:", total, "largest:", largest)
```

Grading: loop that stops at 0 2, count 1, sum 1, largest (works for negatives) 2, "no numbers" case 1, output 1. Starting `largest` at 0 loses the "largest" point if all numbers are negative.